Presence Penalty

Presence Penalty

1) What it is

Presence penalty discourages the model from reusing any token that has **already appeared at least once** (in the prompt and/or the generated text).

- It applies a **flat penalty** per previously seen token—*not* scaled by how many times it appeared.
- Result: **greater vocabulary variety** and fewer echo-y phrases.
- Contrast: **Frequency penalty** scales with *how often* a token has appeared.

---

2) How it works (intuition)

Before sampling, the model adjusts token scores (logits). Presence penalty lowers the score of **every "seen" token** by a fixed amount:

- Heuristic: `adjusted_logit = logit − β * 𝟙(token seen ≥ 1)`
  - `β` = presence penalty strength (e.g., 0.1–1.0)
  - Once a token has appeared, it stays penalized for the rest of the generation.

> Mental model: A flat "don't repeat" nudge after the first use.

---

3) When to use (and when to avoid)

Great fits

- **Creative writing & marketing**: Pushes fresh word choices and varied phrasing.

- **Brainstorming lists**: Reduces identical openings ("Create... Create... Create...").
- **High-temperature outputs**: Helps prevent dull repetition while exploring.

Use carefully / avoid

- **Code, configs, schemas**: Repetition is structural → set near **0**.
- **Legal/medical/translation**: Terms of art must repeat consistently.
- **Structured outputs (tables/JSON)**: Keys and labels must repeat exactly.

---

4) Practical defaults & tuning

Task Type	Presence Penalty	Notes
Factual Q&A, summaries	0.0–0.2	Keep language tight; rely on structure.
Explanations, reports	0.1–0.4	Light variety without destabilizing terminology.
Creative/marketing/ideas	0.3–0.6	Stronger push for novelty; pair with moderate temperature.
Code / JSON / strict terms	**0.0**	Preserve repeated tokens for correctness.

Tips

- Increase in **0.1–0.2** steps; watch for awkward synonym swaps.
- If your provider supports **negative values**, they *encourage* reuse—rarely needed outside strict refrains.

---

5) Interactions with other controls

- **Frequency vs. presence**: Use **presence** to avoid *any* reuse; use **frequency** to throttle *how much* reuse.
- **Temperature / top-p / top-k**: Presence penalty shapes the candidate set; sampling knobs decide which token wins.
- **Max tokens / stop sequences**: Presence penalty doesn't control length—use `max_tokens` and `stop` for boundaries.

---

6) Prompt patterns that pair well

Vary openings across bullets

```
Write 8 bullets. Do not start multiple bullets with the sam
e first word. Vary verbs and sentence length.
```

Keep critical terminology stable

```
Use the exact term “payment authorization hold” whenever ap
plicable. Do not paraphrase this term.
```

Style guardrail

```
Aim for lexical variety. Avoid repeating distinctive adject
ives or phrases unless necessary for clarity.
```

---

7) Troubleshooting

Symptom	Likely Cause	Fix
Important term gets paraphrased away	Presence penalty too high	Lower presence penalty; add a terminology rule in the prompt
Bullets still start the same	Penalty too low or weak instruction	Raise presence penalty; add explicit “no repeated openings” instruction
JSON/code malformed	Penalizing structural tokens	Set presence penalty to 0; enforce structure with JSON/grammar mode

---

8) Copy-paste presets

Balanced prose

```
temperature=0.6, top_p=0.9, presence_penalty=0.2
```

Idea generation / creative

```
temperature=0.8, top_p=0.95, presence_penalty=0.4
```

Code / structured output

```
temperature=0.2, top_p=0.6, presence_penalty=0.0
```

---

9) Key takeaways

- Presence penalty is a **flat “seen-once ⇒ discourage”** control that boosts variety.
- Keep it **low or off** when exact repetition matters (code, legal, translation).
- Combine with **clear prompt instructions** and, if needed, **frequency penalty** to finely balance variety vs. consistency.